

From ASTRA 1.0 to ASTRA 2.0

What changed, what it buys, and what is still unmeasured

Comparison report

17 August 2026

Abstract

ASTRA 1.0 executes one *atomic cycle*: a single decidable proposition, proposed by an ensemble, translated into an executable validator by a different model, reviewed by a third, and run by a deterministic oracle. ASTRA 2.0 keeps that worker unchanged in spirit and adds a layer above it—an append-only evidence ledger, structured proposal portfolios, hard admissibility gates, progressive widening, and resume—so that a research *programme*, not a single question, can be driven to a decision and audited afterwards.

This report states the differences precisely, separates what is already established from what is not, and is explicit about one number that does not yet exist: a head-to-head quality measurement. ASTRA 1.0’s only deposited benchmark data is eight execution smoke runs with zero scientific and zero audit cases, so the comparison everyone expects to see cannot honestly be printed today. The measurement is running; this document is written so the numbers can be dropped into it without rewriting the argument around them.

1 Scope: this is an architecture for commercial models

ASTRA is built to do research with the models that are *commercially available*—subscription-tier CLIs from more than one provider, on a single workstation, under ordinary quota. It is not built on, and makes no claim about, the private or unreleased higher-capability systems the provider companies hold internally.

That constraint is the premise of the design, not a limitation apologised for. Everything distinctive about the architecture follows from it:

- **Independence is bought by heterogeneity, not by scale.** Three providers with different training and different blind spots give a cross-check that a single stronger model cannot give itself.
- **Refutation is mechanical because the models cannot be trusted to self-assess.** The deterministic auditor, the executable validator and the oracle exist precisely because no available model reliably reports when its own argument fails.
- **Budget discipline is structural.** Phase ceilings, downstream reserves and a machine-wide single-cycle lock exist because quota is scarce and shared, not elastic.

The consequence is worth stating plainly: if a substantially stronger model becomes available, ASTRA does not become unnecessary—it becomes cheaper to run, because fewer repair rounds are needed. The architecture is about *where evidence comes from*, and that question does not dissolve with model capability.

2 The two lines side by side

2.1 What 1.0 is

One bounded investigation, end to end: parallel proposals from two providers, cross-critique, synthesis into a consensus conjecture, validator authoring by a third provider, deterministic

preflight, independent review, oracle execution, evidence audit, and a navigator that proposes the next direction. Five status axes are already kept separate. It is a strong single-question engine.

What it does not have is memory across questions. Each cycle begins from a prompt and ends with a verdict and a suggested direction; the minority proposal that lost the synthesis, the branch that was refuted, and the reason either happened are not represented anywhere a later cycle can consult.

2.2 What 2.0 adds

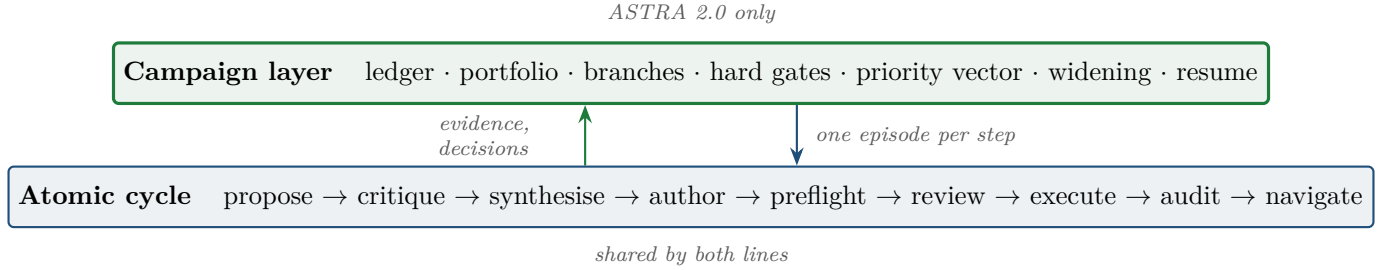


Figure 1: 2.0 does not replace the worker; it closes a loop around it. The upward arrow is the part 1.0 has no place to put.

3 Measured differences

3.1 Code

A drift check compares every shared module three ways—against the common base commit, the current 2.0 tree, and the current production tree.

Classification	Files	Meaning
IN SYNC	60	identical behaviour on both lines
BOTH CHANGED	5	diverged on both sides, needs a decision at merge
2.0 ONLY	15	development evolution not yet in production
PRODUCTION ONLY	0	nothing in production is missing from 2.0

The zero matters: 2.0 is a strict superset of production behaviour plus its own evolution, so a comparison cannot be explained by 2.0 having lost something.

	ASTRA 1.0	ASTRA 2.0
campaign modules (<code>core/campaign_*.py</code>)	0	9
test files	25	54
collected tests	152	469

3.2 Epistemic controls added

Each of these was added because a live measurement exposed the failure it prevents; none is speculative.

Control	Failure it prevents
verdict re-anchoring	a corrected version of a false claim reported as validating the original
decisive vs auxiliary legs	a weak auxiliary check deciding a PASS, or an auxiliary failure printing a scientific FAIL
<code>not_code_at_all</code>	prose handed back to be patched as if it were a damaged script
symbolic cost control	a pointwise claim attacked with full symbolic machinery until memory dies
conjecture fencing	text produced by one agent obeyed as instructions by another
plan-artifact sanitising	a read-only sandbox’s scaffolding leaking into the consensus
thinking budget cap	an output ceiling consumed entirely by internal reasoning
downstream budget reserve	the repair step, which runs last, starved by a proportional rule
machine-wide cycle lock	two checkouts deliberating against one model account

3.3 Ledger guarantees added

These have no counterpart in 1.0 because 1.0 has nothing to guarantee: it does not persist a research record.

- append-only events with a canonical-JSON SHA-256 chain;
- atomic checkpoints, and a truncated tail detected and archived rather than silently repaired;
- content-addressed frozen resources, so editing an input is detectable;
- a replayed cycle refused rather than recorded as a second observation;
- a result bound to the request’s objective, so a cycle that answered a different question cannot become evidence;
- human interventions recorded as first-class decisions with an actor, rather than silently changing the record.

4 What is not measured yet

4.1 The head-to-head number does not exist

The figures usually quoted for ASTRA—false acceptance 0.0, false rejection 0.0, strict accuracy 0.696, audit recall 1.0, sound-validator false alarm 0.0, over 43 cases—were produced by the *2.0 code* running production’s *configuration*. They measure 2.0.

ASTRA 1.0’s own deposited benchmark data is eight runs, all smoke tier, whose summaries read `scientific: {cases: 0}` and `audit: {cases: 0}`. The only quantity ever measured on that line is execution reproducibility (verdict accuracy 1.0, cross-run agreement 1.0, $n = 8$).

Standard tier, 43 cases	ASTRA 1.0	ASTRA 2.0
scientific cases	<i>pending</i>	23
false acceptance	<i>pending</i>	0.0
false rejection	<i>pending</i>	0.0
strict accuracy	<i>pending</i>	0.696
audit: critical defect recall	<i>pending</i>	1.0
audit: false alarm, sound code	<i>pending</i>	0.0
execution: verdict accuracy	1.0 ($n=8$, smoke)	1.0

The measurement is running on the production line with the same suite, tier and oracle. Two caveats belong with it when it lands: one audit case differs between the checkouts by a fixture correction, and—more importantly—this benchmark exercises the atomic cycle only.

4.2 The benchmark cannot see the main contribution

This is the honest limit of any 1.0/2.0 comparison. The quality benchmark scores single cycles. The campaign layer is not exercised by it at all, and 1.0 has no campaign layer to compare against, so on that axis the difference is not a number but a presence.

What the campaign layer buys has to be argued from what it makes possible, and so far the evidence is qualitative but concrete: in live use it recorded refutations with exact witnesses against a persistent claim set, promoted a branch on negative evidence under a deterministic rule, refused to widen on an episode that never ran, retracted a contaminated record without rewriting history, and stopped a validator whose conclusions came from undefined variables. None of those events has a place to happen in 1.0.

4.3 And a first comparative signal that is not favourable

Within 2.0, a preregistered ablation of the reflective loop against a matched linear control did not meet its own interpretability criterion—operational failure stayed far above the 0.2 threshold fixed in advance—and the raw counts leaned against the reflective arm. That result is reported in the architecture report rather than hidden here. It concerns the *policy* inside 2.0, not the 1.0/2.0 difference, but anyone reading this document should know it exists.

5 Why the extra structure is worth it

Three arguments, in decreasing order of how well they are evidenced.

It makes negative results usable. Most research time is spent pruning. In 1.0 a refutation is a verdict that scrolls past; in 2.0 it closes a branch, promotes an alternative under a stated rule, and stays in a record that the next cycle consults. Pruning becomes cumulative instead of repeated.

It makes the record auditable by someone who was not there. Every claim carries a validator, its hash, the model that wrote it, the reviewer’s objection, the oracle’s output, and the decision that followed. That is the difference between a transcript and evidence, and it is what makes an LLM-driven result quotable at all.

It makes the system’s own failures visible. The defects found in 2.0—a reviewer refusal recorded as breakage, a cached replay minted as evidence, a foreign result accepted through a runner hook—are defects 1.0 cannot exhibit, because it never acts on its own signals. Acting on them is where the value is, and also where the risk is; the layer that creates the risk is the same layer that lets it be caught.

6 Position

ASTRA 2.0 is a strict superset of 1.0 in behaviour, with 9 new modules, three times the test coverage, nine epistemic controls each traceable to a measured failure, and a persistent evidence ledger with guarantees 1.0 has no need for. It is demonstrably more *careful*.

Whether it is more *productive* per unit of quota is not established, and this document declines to assert it until the paired benchmark lands and a blinded expert scoring of trajectories exists. The architecture report states the same thing about its internal ablation. Claiming otherwise would be exactly the failure mode the whole system is built to prevent.

Note on scope. Both lines run on commercially available subscription models from three providers on one workstation (Section 1). Nothing here is a claim about frontier or unreleased systems, and no result depends on privileged model access.